A

MINI PROJECT REPORT

ON

"TrackMyLogs"

Submitted by

Diya Mehta (22IT489)

Rudra Prajapati (23IT604)

Daksh Khungla (23IT605)

Prof. Prachi Shah
(Faculty Guide)

Information Technology Department

Birla Vishvakarma Mahavidyalaya Engineering College

(An Autonomous Institution)

AY: 2024-25, Semester II

Birla Vishvakarma Mahavidyalaya Engineering College
(An Autonomous Institution)
Information Technology Department
AY: 2024-25, Semester II

CERTIFICATE

This is to certify that project entitled with **"TrackMyLogs"** has been successfully carried out by **Rudra Prajapati (23IT604), Daksh Khungla (23IT605), Diya Mehta (22IT489)** for the subject of **3IT31- Mini Project** under my guidance during the academic year 2024-25, Semester II.

The Mini Project work carried out by the students of 6th semester is satisfactory.

Date: 16/04/2025

Prof. Prachi Shah	**Dr. Nilesh B. Prajapati**	**Dr. Keyur N. Brahmbhatt**
Faculty Guide	Course Coordinator	Head of the Department
IT Department	IT Department	IT Department
BVM	BVM	BVM

Acknowledgement

We would like to express our heartfelt gratitude to our guide, **Prof. Prachi Shah**, for her continuous support, valuable guidance, and encouragement throughout the development of our mini project titled **"TrackMyLogs."** Her insights were instrumental in helping us complete this project successfully.

We also extend our sincere thanks to **Dr. Nilesh B. Prajapati**, Course Coordinator, and **Dr. Keyur N. Brahmbhatt**, Head of the Department, for providing us with the necessary resources and a conducive academic environment. We are thankful to all the faculty members of the Information Technology Department for their support, and to our families and friends for their encouragement. Lastly, we appreciate the teamwork, dedication, and cooperation among ourselves—**Diya Mehta (22IT489), Rudra Prajapati (23IT604), and Daksh Khungla (23IT605)**—which played a vital role in the successful completion of this project.

List of Figures

List of Tables

Abstract

The increasing reliance on system logs for monitoring, diagnostics, and security analysis has created a need for efficient, user-friendly, and secure log management solutions. **TrackMyLogs** is a mini project aimed at addressing this need by providing a cross-platform tool that collects system logs from Windows machines, transfers them securely to a Django-based web application, and presents them through an intuitive interface for analysis and visualization. The system includes modules for log generation, secure log transfer using HTTPS, user authentication with role-based access control, and analytical tools for trend identification and report generation. Logs are compiled into a PDF and sent to the registered user's email, enhancing accessibility and usability. The backend integrates SQLite for storage and Redis for temporary data handling, while the frontend is developed using modern web technologies. This project demonstrates a modular, scalable, and secure approach to log management, making it a valuable tool for both individual users and organizations aiming to monitor system behavior and maintain operational transparency.

Index

Chapter 1: Introduction

1.1 Aim of the Project

TrackMyLogs is a system monitoring tool designed to facilitate the collection, transfer, and visualization of logs from a Windows application to a web-based platform. The application aims to provide users with a user-friendly interface for managing logs, enabling them to gain insights and make informed decisions based on log data.

1.2 Project Scope

- The collected logs will be compiled into a PDF and delivered to the registered email ID upon completion.
- Django is used for creating software download systems and managing authentication keys.
- Integration of secure data transfer protocols (HTTPS).
- User authentication and role-based access control (RBAC).
- Analytical tools for identifying trends and generating reports.

1.3 Project Objective

- To develop a Windows application that allows users to generate logs easily.
- To implement a secure and efficient method for transferring logs to a web server.
- The logs generated will be converted into a PDF and sent to the registered email ID at the end.
- To provide users with tools for log analysis, including reporting and visualization features.

1.4 Project Modules

1. Log Generation Module.
2. Log Transfer Module.
3. Log Display Module
4. User Authentication Module.
5. Log Analysis Module.
6. Key Management Module.

1.5 Project basic requirements

1.5.1 Hardware Requirements

Client Machine: Windows 10 or later, minimum 4 GB RAM, minimum 500 MB free disk space.

1.5.2 Software Requirements

- Development Environment: Python 3.x, Django framework, SQLite database, redis,Windows application development tools (e.g., PyQt or Tkinter).
- Web Browser: Latest version of Chrome, Firefox, or Edge for accessing the web application

Chapter 2: Analysis, Design Methodology and Implementation Strategy

2.1 Comparison of Existing Applications with your Project / Literature review.

Comparison of Existing Applications with your Project:

Features	TrackMyLogs	Splunk	Loggly
Log Generation	Yes	Yes	Yes
Log Display	Yes	Yes	Yes
User Authentication	Yes	Yes	Yes
User Analysis	Yes	Yes	Yes
User Activity Tracking	Yes	No	No
Multi User Support	Yes	Yes	Yes
Automated Log Cleanup	Yes	No	No
Mobile Support	No	Yes	Yes

Table 2.1 Comparison Table of Existing Application

Literature Review:

2.1.1 Title: "Log Management: Challenges and Solutions"

Abstract:

This paper discusses the various challenges faced in log management, including data volume, format inconsistencies, and security concerns. It presents solutions such as standardized log formats and automated log analysis tools to enhance the efficiency of log management systems.

Summary:

The authors highlight the importance of adopting standardized log formats to improve data consistency and facilitate easier analysis. They also emphasize the need for automated tools that can process large volumes of log data, reducing the manual effort required for log management. The paper concludes with recommendations for organizations to implement best practices in log management[1].

2.1.2 Title: "Secure Log Transfer Protocols: A Comparative Study"

Abstract:

This study evaluates various secure log transfer protocols, focusing on their effectiveness in protecting sensitive log data during transmission. The authors compare HTTPS, SFTP, and other protocols, providing insights into their strengths and weaknesses.

Summary:

The findings indicate that HTTPS is the most widely adopted protocol for secure log transfer due to its ease of implementation and strong encryption capabilities. The study also discusses the importance of user-configurable endpoints for flexibility in log management systems. The authors recommend adopting HTTPS as a standard for secure log transmission[2].

2.1.3 Title: "User -Centric Design in Log Management Systems"

Abstract:

This research explores the significance of user-centric design in log management systems. It examines how user interface (UI) and user experience (UX) impact the effectiveness of log analysis and management.

Summary:

The authors argue that an intuitive UI and UX are critical for enabling users to efficiently navigate and analyze log data. They provide case studies demonstrating how well-designed interfaces can lead to improved user satisfaction and faster decision-making. The paper advocates for incorporating user feedback into the design process to create more effective log management tools[3].

2.1.4 Title: "Analyzing Log Data: Techniques and Tools"

Abstract:

This paper reviews various techniques and tools for analyzing log data, including statistical methods, machine learning algorithms, and visualization tools. It discusses the advantages and limitations of each approach.

Summary:

The authors highlight the growing importance of advanced analytical techniques in log management, particularly in identifying trends and anomalies. They provide a comparative analysis of popular tools used for log analysis, emphasizing the need for integration with existing log management systems. The paper concludes with recommendations for selecting appropriate tools based on organizational needs[4].

2.1.5 Title: "Role-Based Access Control in Log Management"

Abstract:

This study investigates the implementation of role-based access control (RBAC) in log management systems. It discusses the benefits of RBAC in enhancing security and managing user permissions effectively.

Summary:

The authors demonstrate that RBAC can significantly improve the security of log management systems by ensuring that only authorized users have access to sensitive log data. They provide guidelines for implementing RBAC, including defining user roles and permissions. The paper emphasizes the importance of balancing security with usability to maintain an effective log management process[5].

2.2 Project Feasibility Study

2.2.1 Financial Feasibility

1. **Operational Costs:** Ongoing expenses will include server hosting fees, maintenance costs, and potential marketing expenses to promote the application.

2. **Revenue Projections:** The application may generate revenue through subscription models, licensing fees, and consulting services related to log management.

3. **Return on Investment (ROI):** A cost-benefit analysis will be conducted to compare total costs against expected financial benefits, estimating the time required to break even.
4. **Market Demand:** An analysis of market demand for log management solutions will help justify the financial investment and guide pricing strategies.

2.2.2 Technical Feasibility

1. **Technology Stack:** The application will utilize Python and Django for backend development, with MySQLite for database management.
2. **System Architecture:** A client-server model will be implemented, allowing to generate logs and a web application for analysis.
3. **Integration Capabilities:** The application will support secure data transfer using HTTPS and may integrate with existing logging frameworks.
4. **Resource Availability:** The development team possesses the necessary skills in the chosen technologies, ensuring effective project execution.
5. **Development Tools:** Availability of required development tools and environments will be assessed to facilitate the development process.

2.2.3 Operational Feasibility

1. **User Adoption:** The application is designed to be user-friendly, which should facilitate adoption among users with varying technical skills.
2. **Training Requirements:** Minimal training will be required for users, as the application will include intuitive interfaces and help documentation.
3. **Compliance and Standards:** The application will adhere to relevant data protection regulations and industry best practices for software development.
4. **Risk Assessment:** Potential risks associated with project execution will be identified, along with strategies to mitigate these risks

2.3 Project Timeline chart

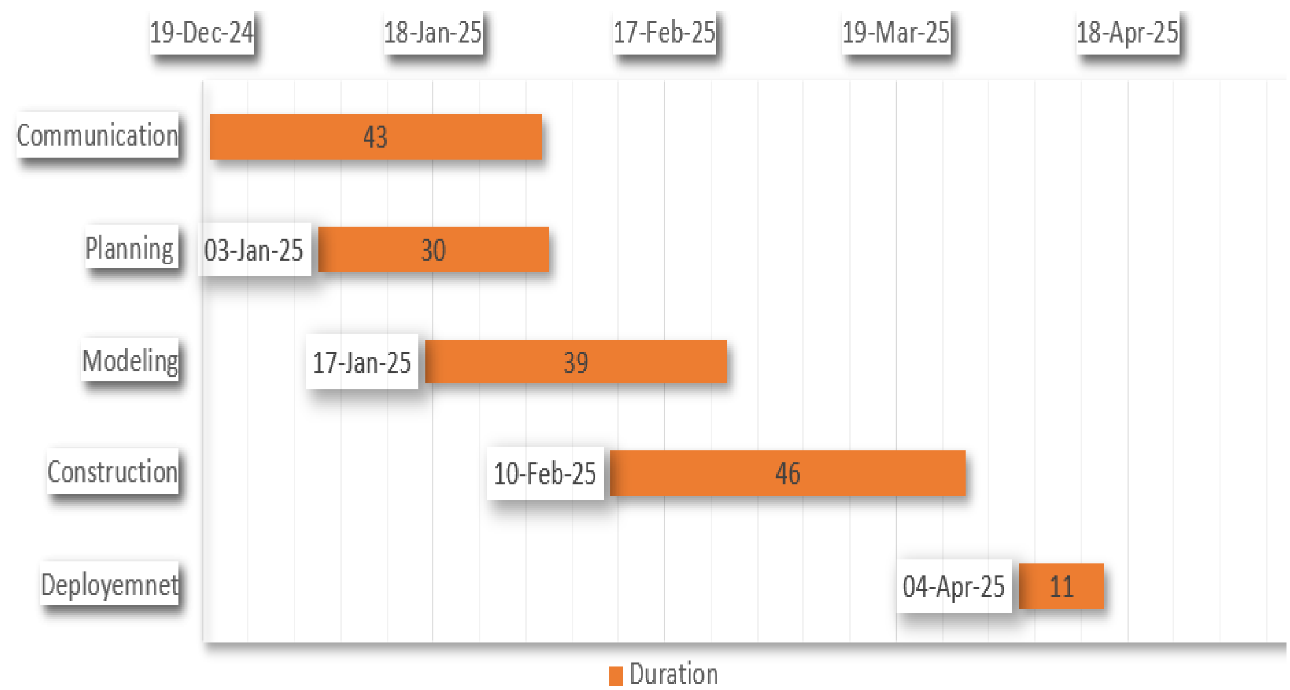

Figure 2.1 Project Timeline chart

2.4 Detailed Modules Description

1. **Log Generation Module:**

   This module operates on the client-side (Windows system) and is responsible for capturing real-time system and application events. It utilizes the list of python modules to access Windows Event Logs and generate structured logs, including application usage, system events, and optionally network activity. The logs are formatted in a consistent structure for smooth parsing and further processing. This module also includes filters to select relevant logs and avoid excessive data generation.

2. **Log Transfer Module:**

   Once the logs are generated, this module handles their secure and efficient transmission to the Django-based web server. HTTPS protocol is used to encrypt and protect data during transit. A lightweight REST API is designed to receive log files from client machines. The module also ensures that logs are temporarily stored in Redis until they are processed or saved permanently, preventing data loss during network interruptions.

3. **Log Display Module:**

   After the logs are received by the server, this module manages their visualization in the web application. It fetches and processes the logs from the backend, converting them into user-readable formats such as tables and charts using tools like Matplotlib and Chart.js. It also supports search, filtering, and pagination to enhance the user experience when browsing large volumes of log data. Users can request a PDF compilation of the logs, which is automatically generated and emailed to them.

4. **User Authentication Module:**

   This module secures the application by providing functionalities for user registration, login, and session management. Built using Django's authentication system, it supports password hashing, user verification, and token-based session control. Role-Based Access Control (RBAC) is implemented to restrict access to certain features (e.g., only staff users can generate tokens or view analytics). This ensures data privacy and personalized experiences.

5. **Log Analysis Module:**

   To derive actionable insights from raw log data, this module performs automated analysis such as frequency tracking, error rate detection, and anomaly identification. It supports graphical representations for system behavior trends, active time graphs, and suspicious activity reports. In future iterations, this module may incorporate ML-based anomaly detection for advanced analytics.

6. **Key Management Module:**

To ensure that each user securely interacts with their own log environment, this module manages authentication keys. Upon request (post-login), a unique key is generated for each user. The key is stored in the backend (SQLite) and associated with that specific user account. This module ensures that only one key per user is generated to avoid misuse.

2.5 Project SRS

2.5.1 Use Case Diagrams

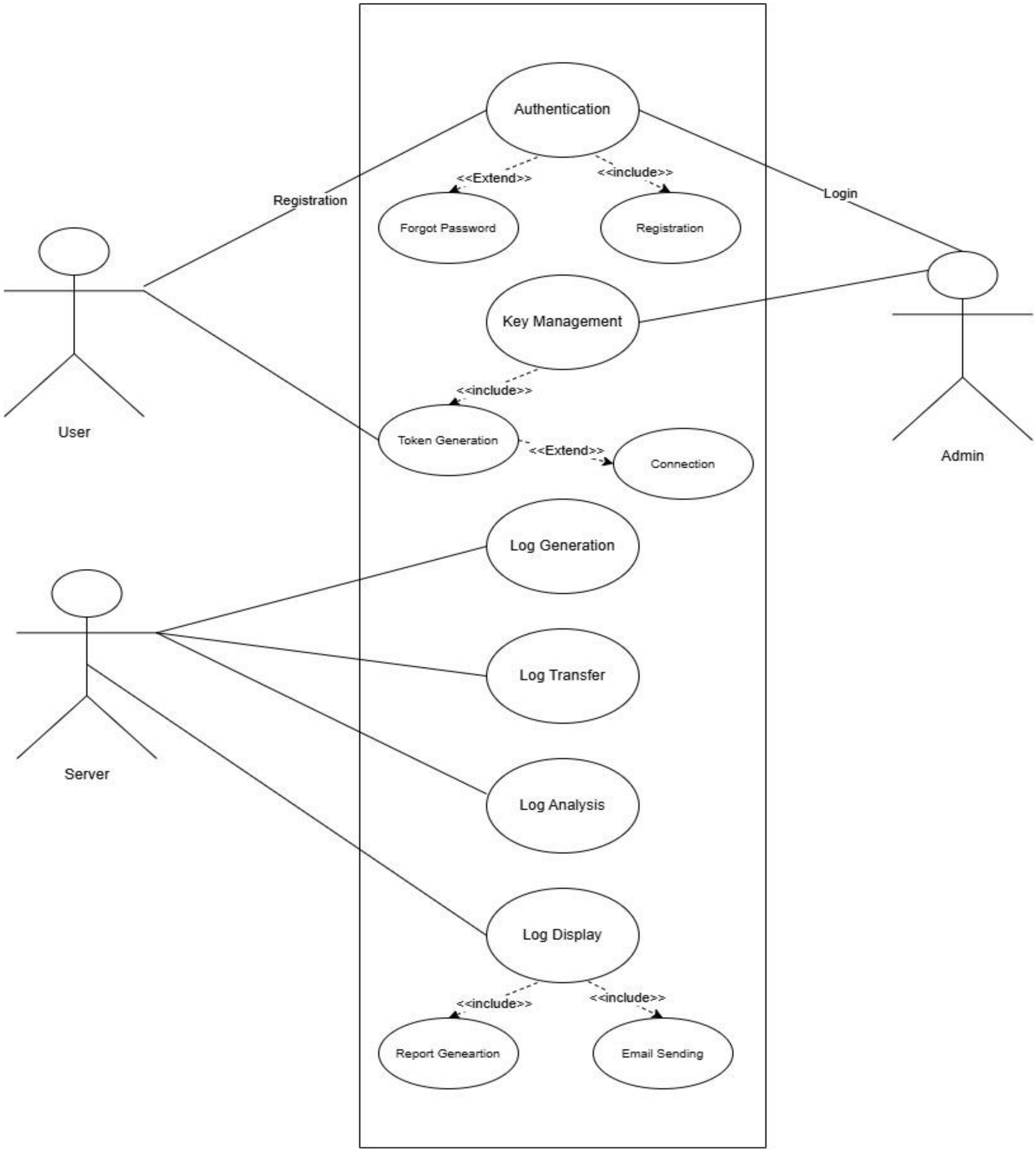

Figure 2.2 Use Case Diagram

2.5.2 Data Flow Diagrams

Level 0:

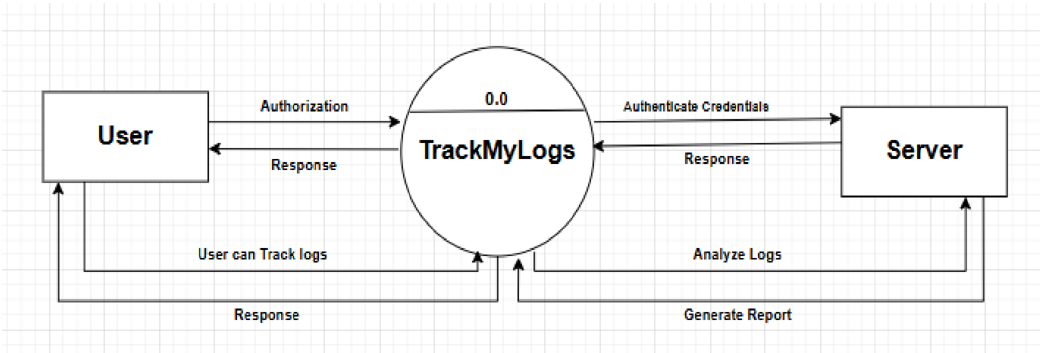

Figure 2.3 DFD Level 0

Level 1:

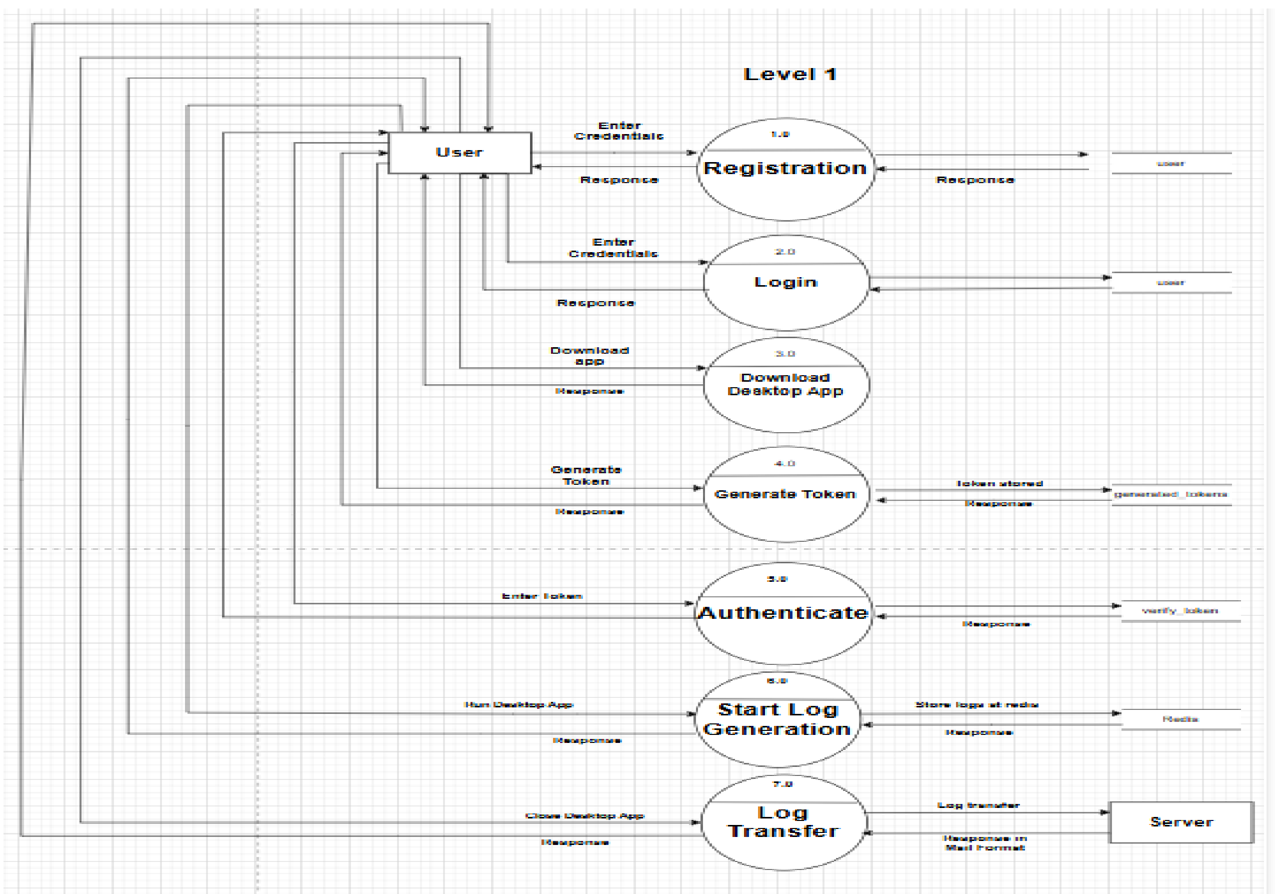

Figure 2.4 DFD Level 1

2.5.3 Entity Relationship Diagrams

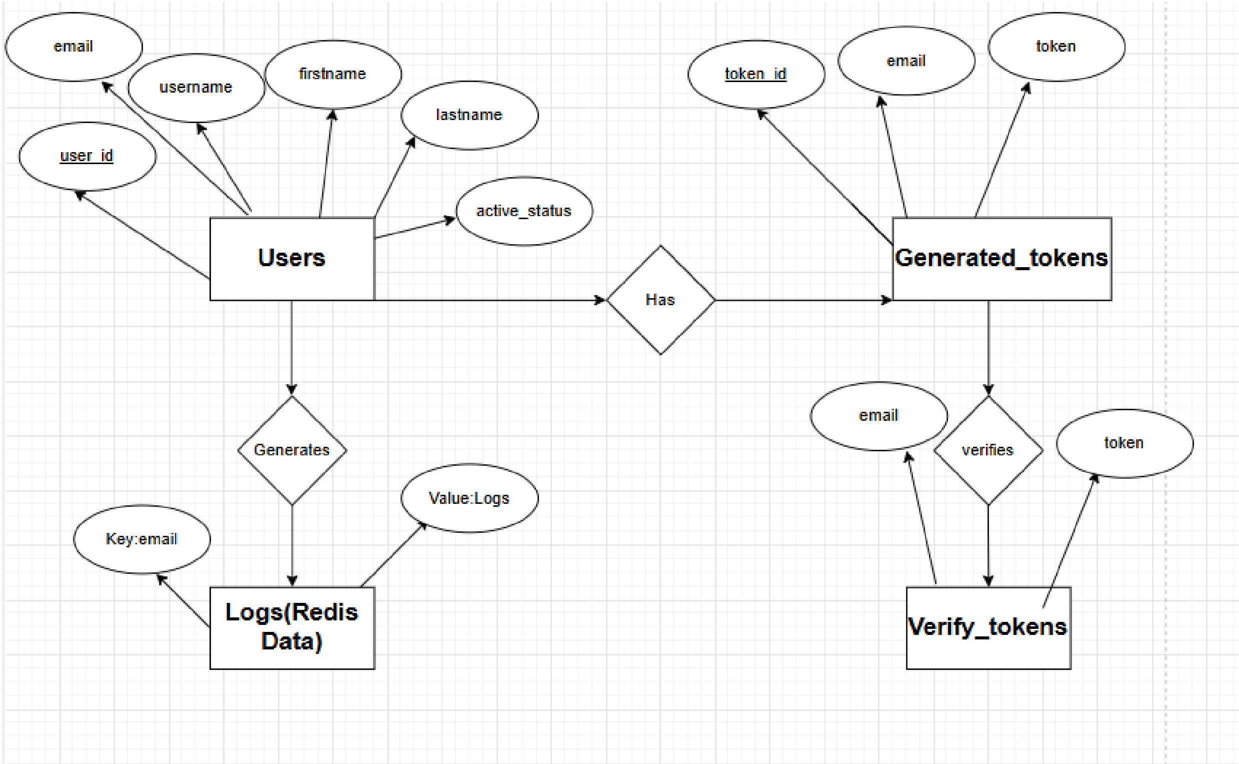

Figure 2.5 ER Diagram

2.5.4 Event Trace Diagram

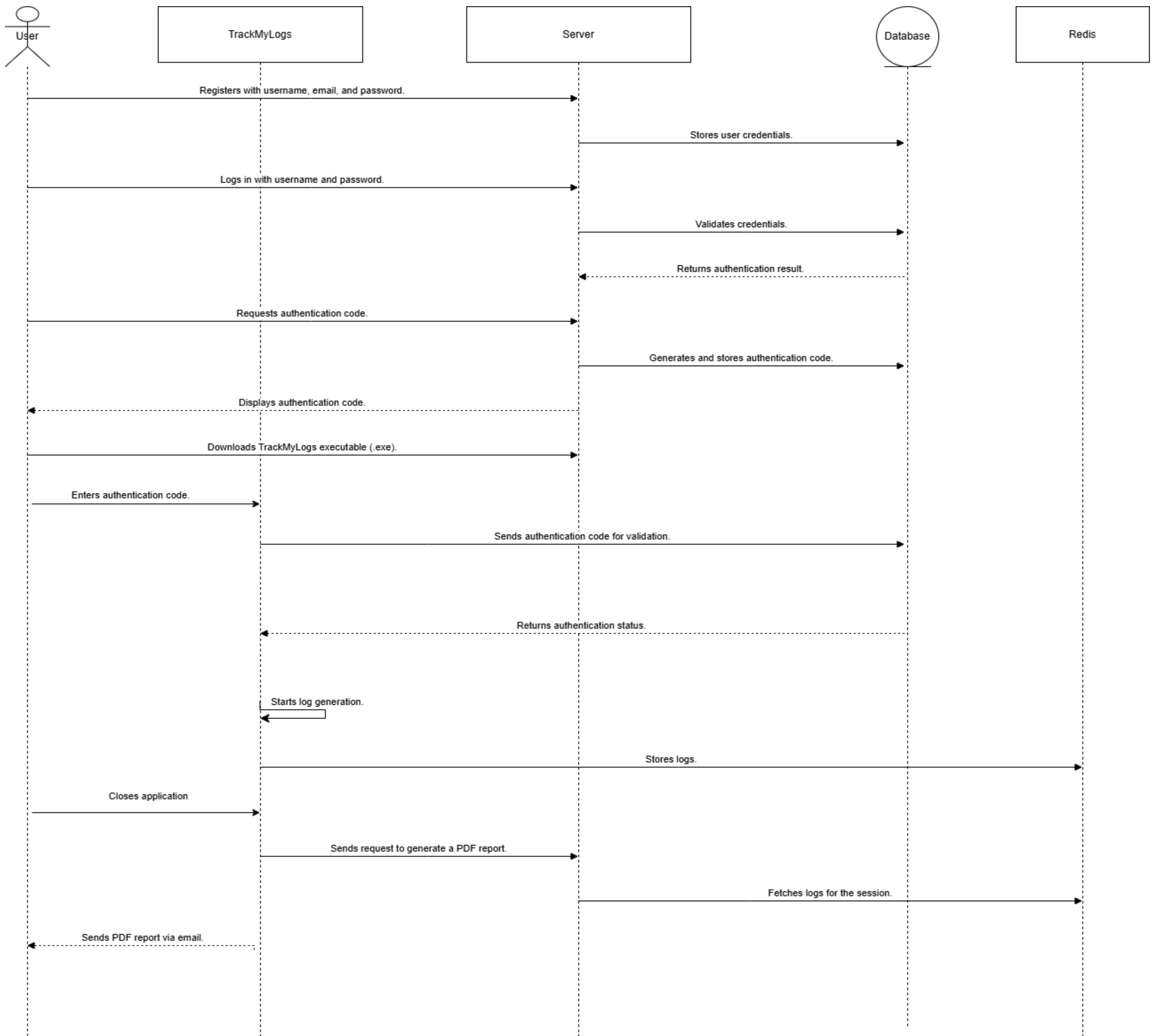

Figure 2.6 Event Trace Diagram

2.5.5 State diagram for Web Application

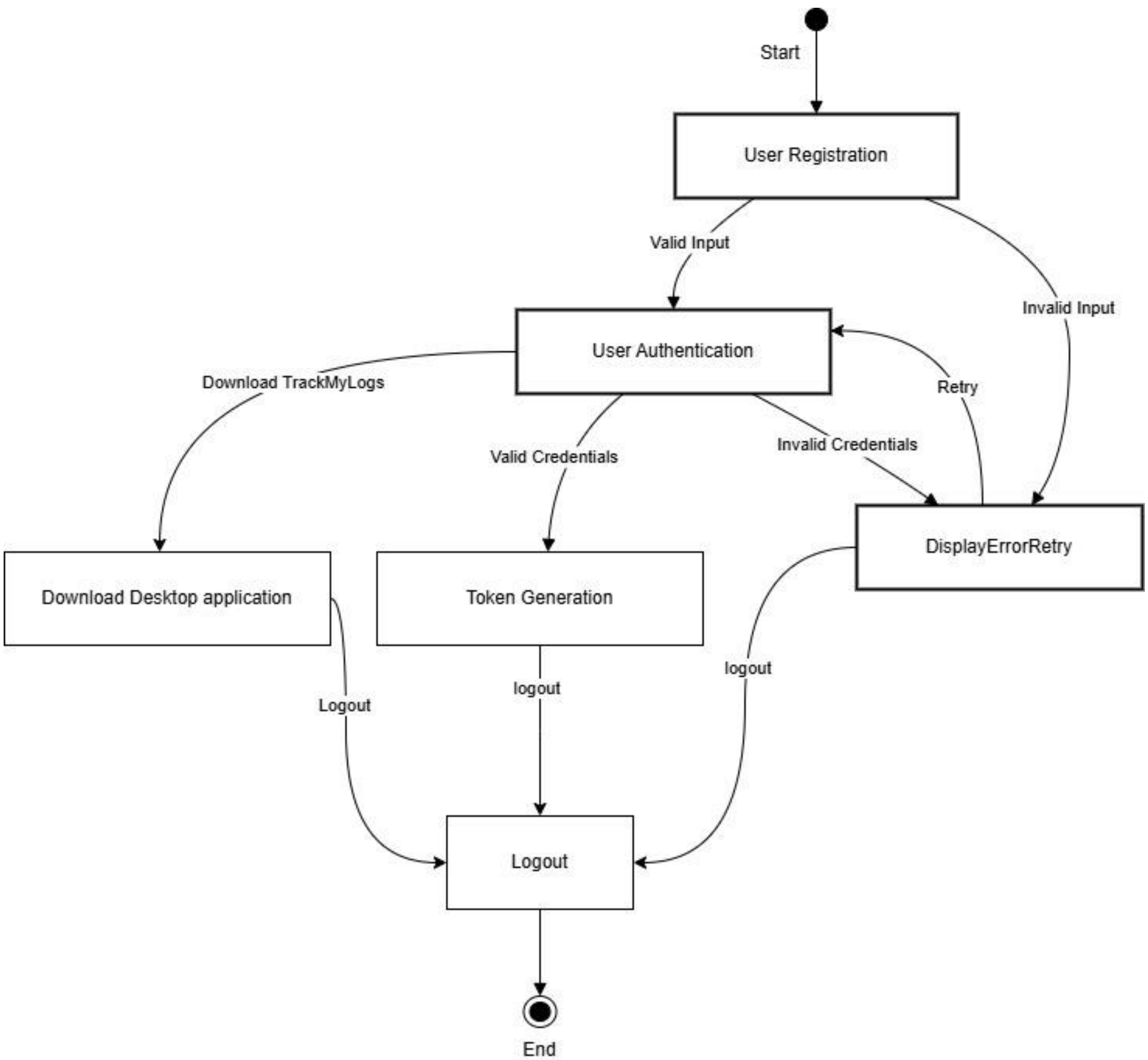

Figure 2.7 State Diagram for Web Application

2.5.6 State diagram for Desktop Application

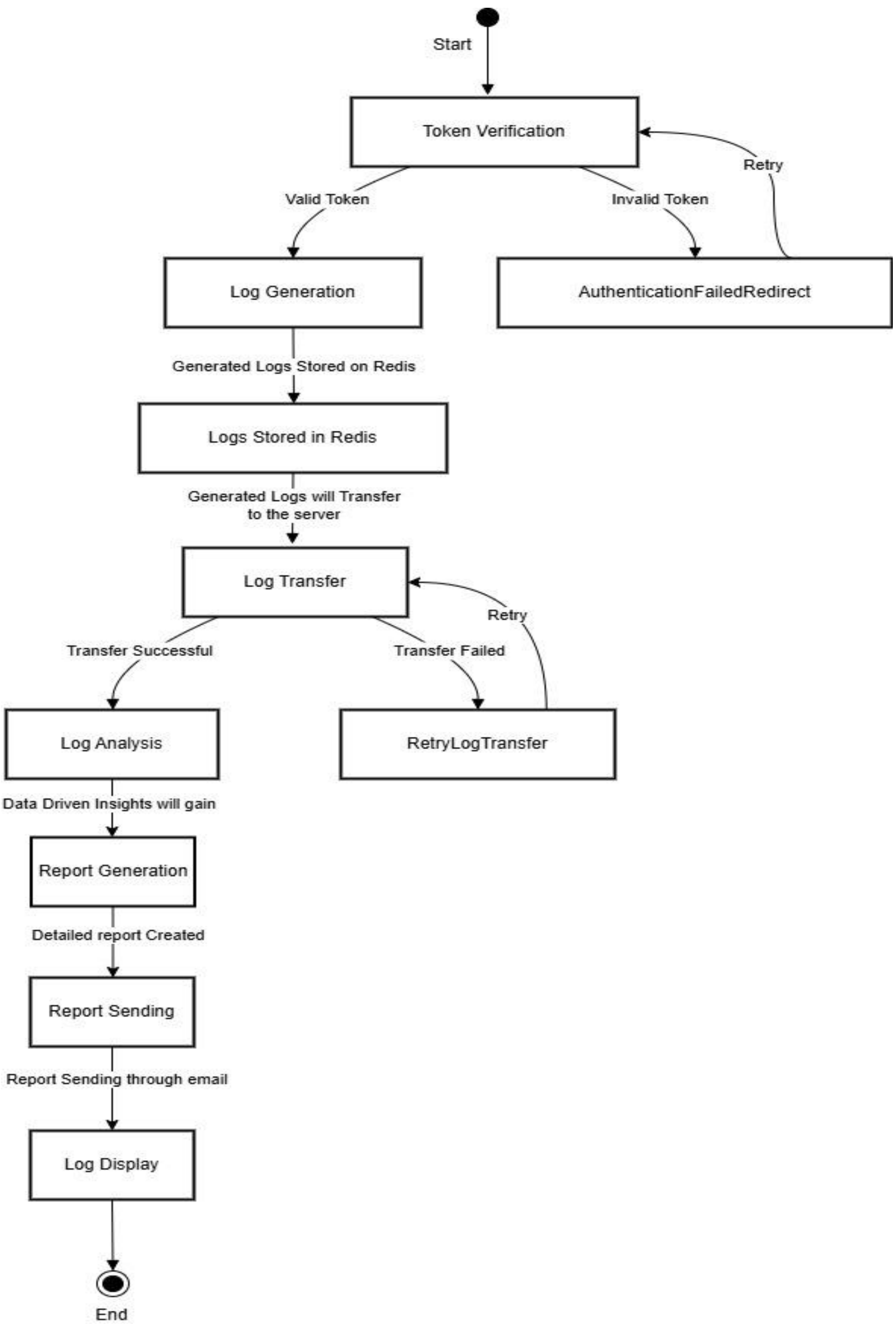

Figure 2.8 State Diagram for Desktop Application

2.5.7 Class diagram

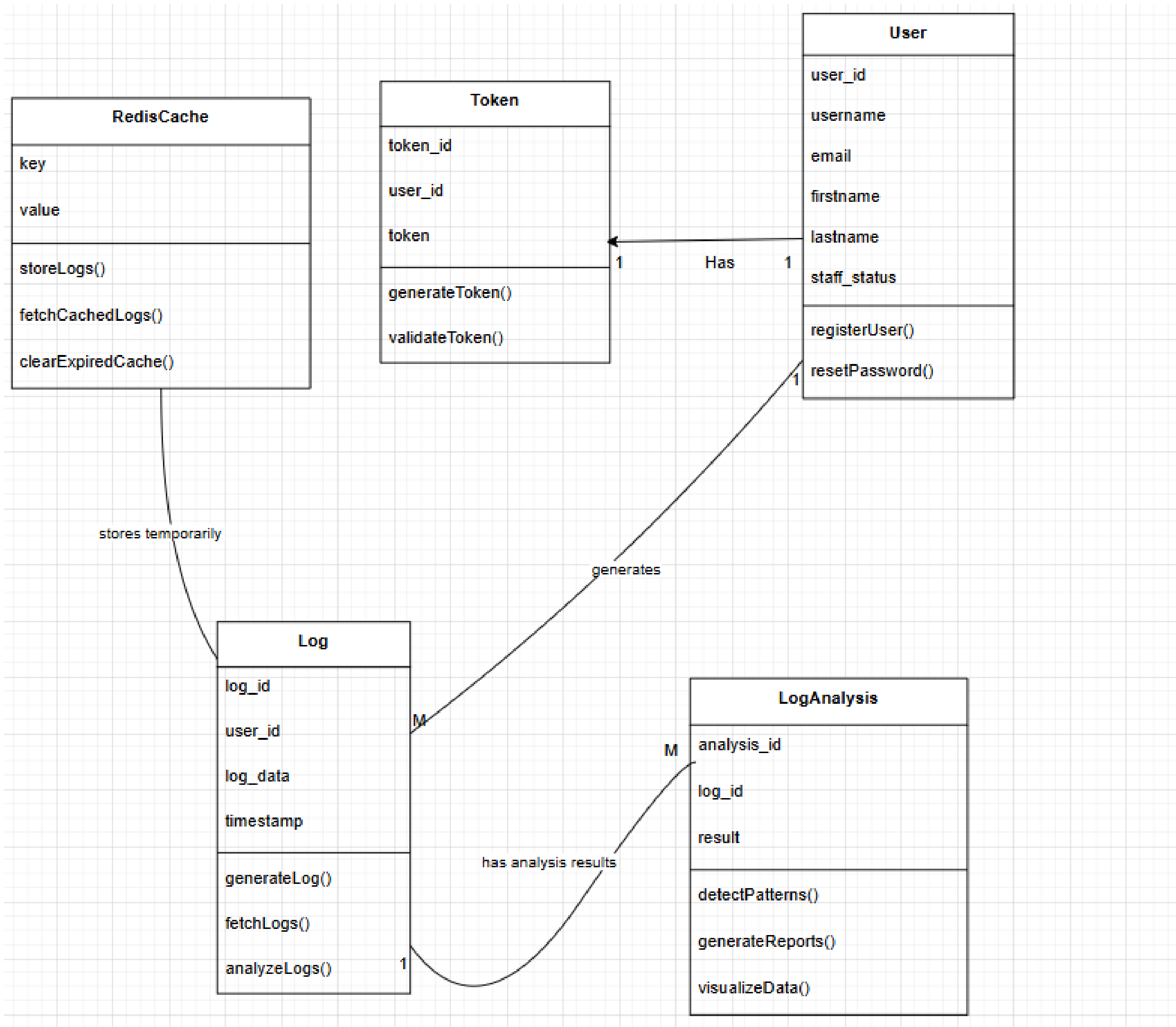

Figure 2.9 Class Diagram

2.6 Database design and Normalization

SQLLite Tables:

2.6.1 Users

Column Name	Data Type	Description
user_id	Int	Unique identifier for each user
Username	Varchar	Username of the user
Email	Varchar	Email address of the user
Firstname	Varchar	First name of the user (nullable)
Lastname	Varchar	Last name of the user (nullable)
Staff_status	Varchar	Indicates if the user is a staff member

Table 2.2 Users Table Schema

2.6.2 Generated_Tokens

Column Name	Data Type	Description
Id	Int	Unique identifier for each token
Username	Varchar	Username of the user
Email	Varchar	Email address of the user
Token	Varchar	The Token of the user

Table 2.3 Generated_token Table Schema

2.6.3 Redis Data

Key	Value	Description
Emails	Logs	Stores log data for specific events or errors.

Table 2.4 Redis Key-Value data structure

2.7 Database Relation Diagram

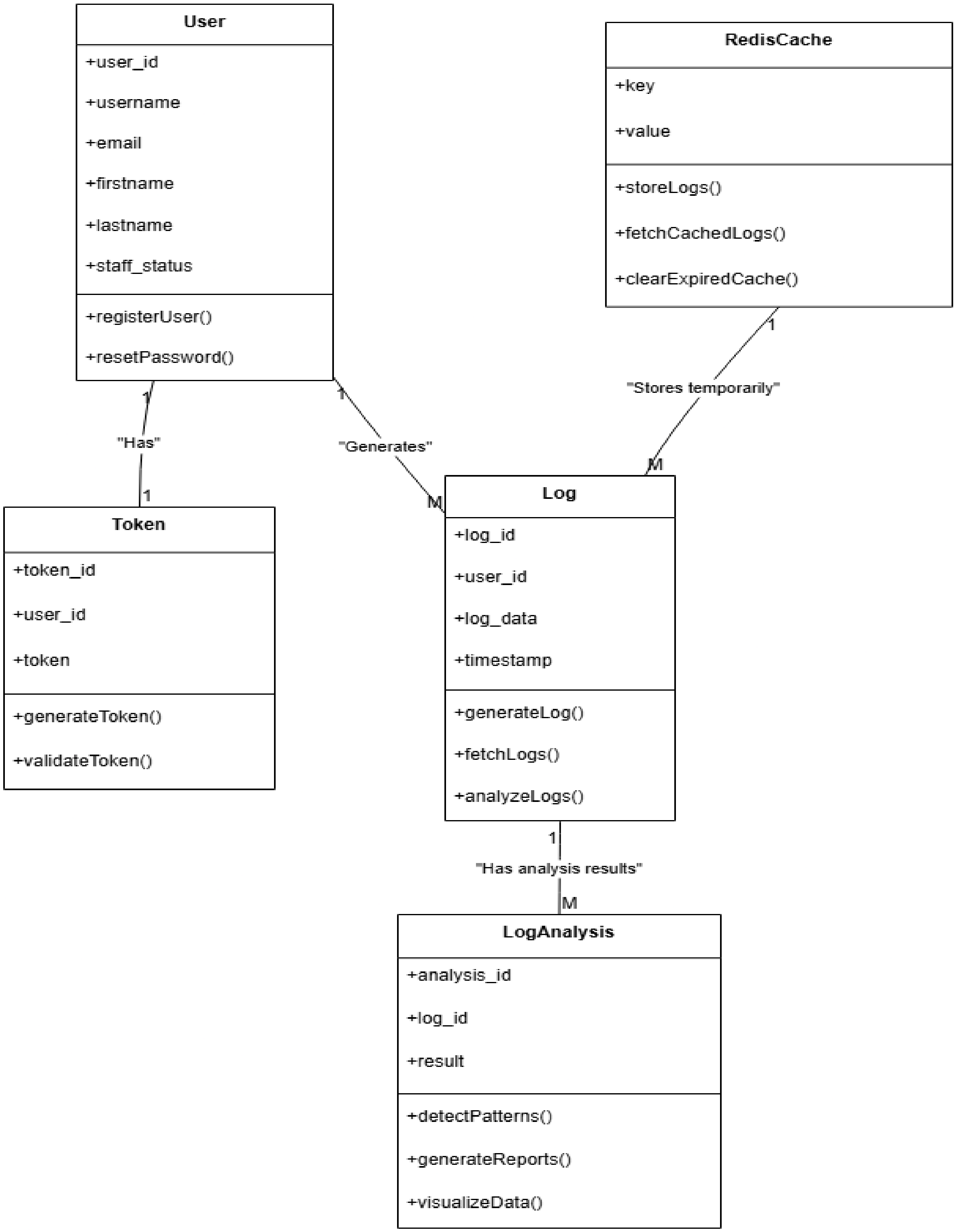

Figure 2.10 Database Relation Diagram

2.8 Template Design

2.8.1 Login Signup Page:

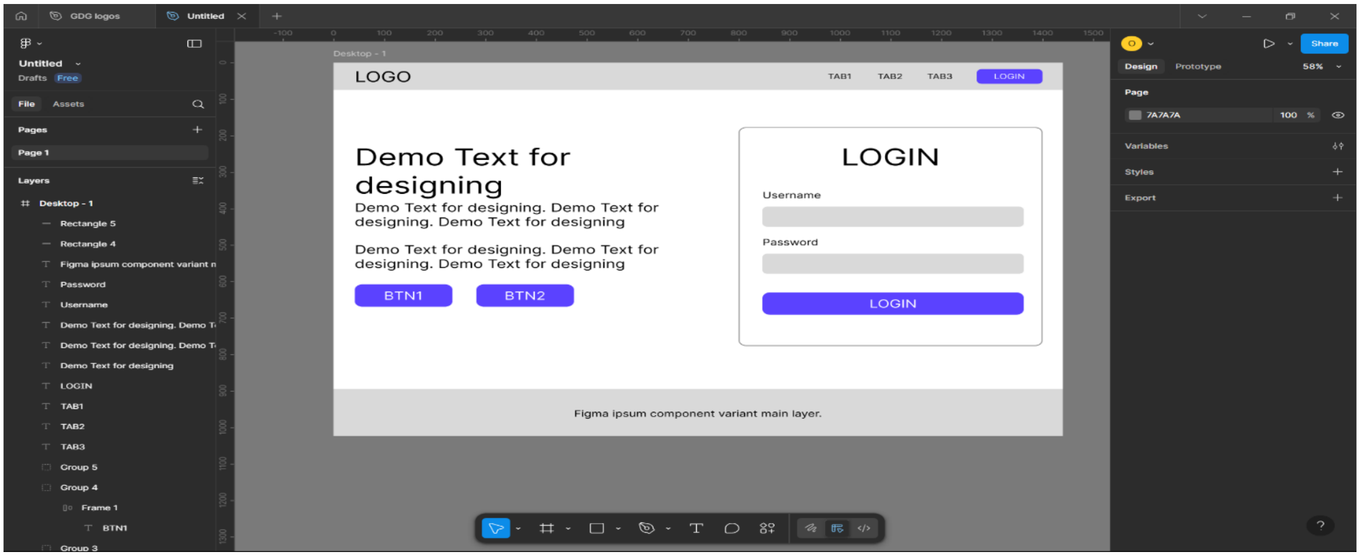

Figure 2.11 Template Design Preview 1

- Initial Design Template for the login/Signup page to get the user reviews.

2.8.1 Home Page:

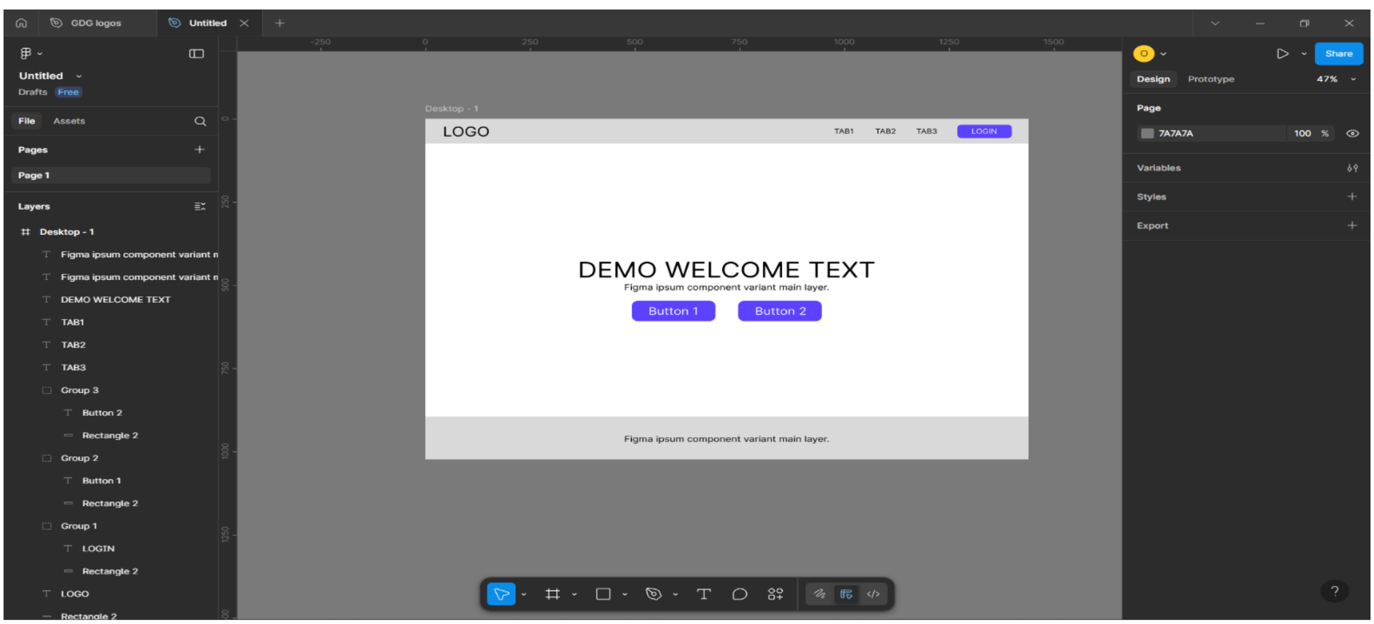

Figure 2.12 Template Design Preview 2

- Initial Design Template for the Home Page to get the user reviews.

Chapter 3: Implementation and Testing

3.1 Software and Tools

The following software and tools were used during the development, testing, and documentation of the TrackMyLogs project:

1. **Visual Studio Code**
   Served as the primary code editor for developing the application using **Python (Django framework)** along with **HTML, CSS, and JavaScript** for the frontend. Its integrated terminal and extensions greatly enhanced productivity.
2. **Web Browsers (Google Chrome)**
   Used extensively for testing the Django web application, verifying UI functionality, debugging with developer tools, and accessing documentation and APIs.
3. **Draw.io**
   Utilized to design and generate key system diagrams such as the **Entity-Relationship Diagram**, **Class Diagram**, and **Database Relation Diagram**, which visually represent the backend structure and logic.
4. **Microsoft Word**
   Used to prepare and format the final project documentation, including the report, system design documentation, and technical descriptions.
5. **SQLite3 (with Django ORM)**
   Acted as the backend **relational database** for storing user data, logs, analysis results, and authentication tokens. Integrated seamlessly with Django's Object Relational Mapper (ORM) for efficient database operations.

3.2 User Interface and Snapshot

3.2.1 Desktop App

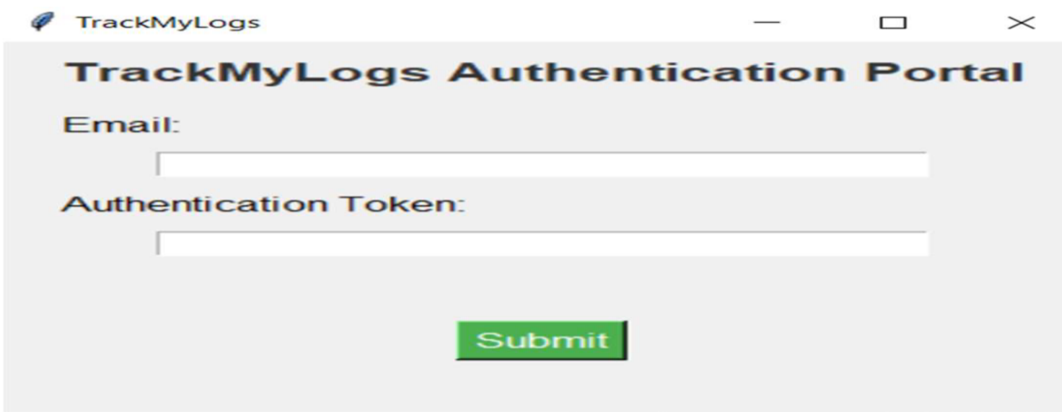

Figure 3.1 Desktop App

- Upon launching the **desktop application**, the user is prompted to authenticate using the token provided on the web application. Once authenticated, the application begins monitoring and collecting logs in real-time from the user's Windows system. These logs include application usage, system events, and optionally, network activity. The application runs silently in the background, ensuring minimal disruption to the user's workflow while maintaining active log generation.

3.2.2 Login Page

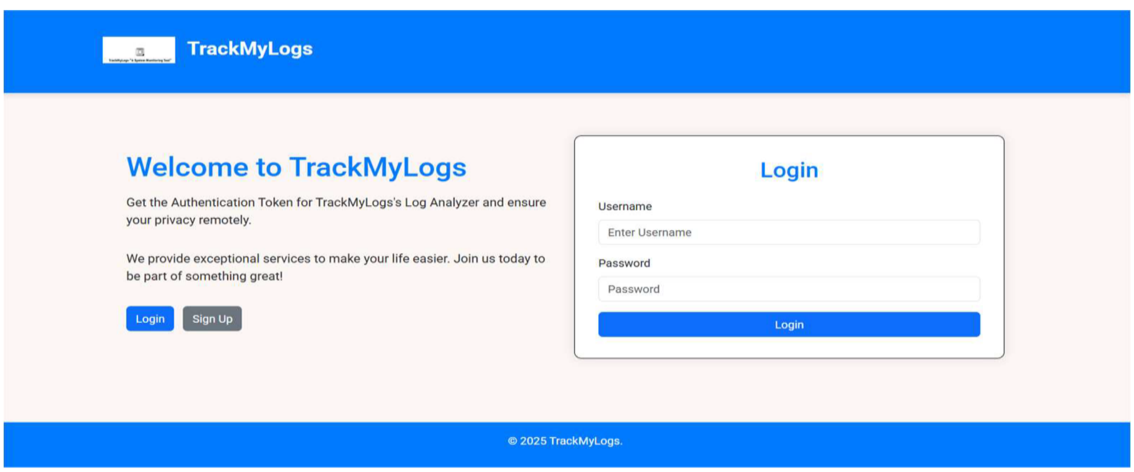

Figure 3.2 Login Page

- The **web application** opens with a **Login Page** where users enter their registered email and password to access their dashboard. For new users, a registration link is provided. All login data is validated through Django's secure authentication system, and upon success, the user is redirected to their personalized profile page.

3.2.3 Profile Page

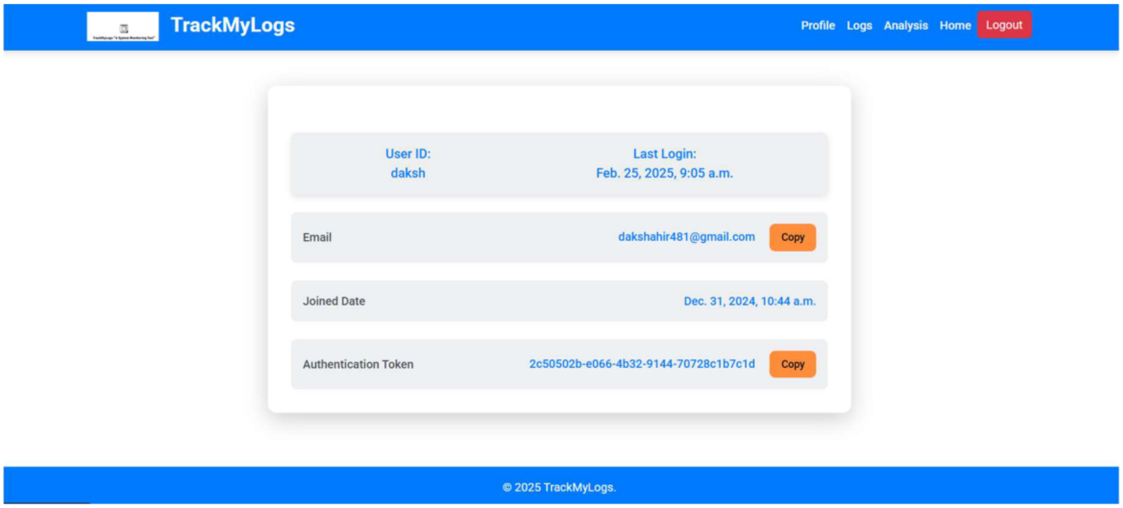

Figure 3.3 Profile Page

- On the **Profile Page**, users can view their account information, including email, username, and whether they hold staff privileges. A core function on this page is the ability to generate or view their **Authorization Token**, which is required to authenticate the desktop application. If the token has already been generated, it is displayed and can be copied to the clipboard.

3.2.4 Logs Page

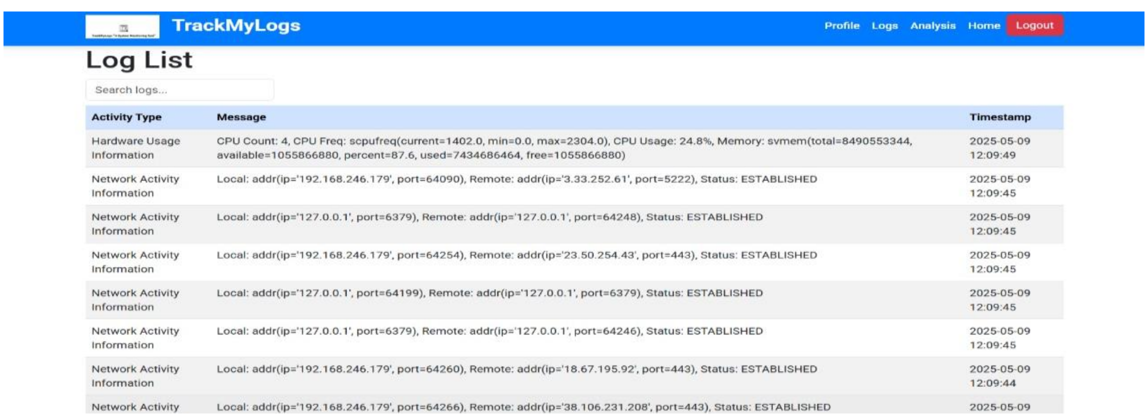

Figure 3.4 Logs Page

- After authentication, users can navigate to the **Logs Page**, where the logs collected by the desktop application are displayed in an organized tabular format. Users have access to powerful **filtering options**. Users can also perform searches and sort the data dynamically. This ensures efficient navigation even when dealing with large datasets.

3.2.5 Analysis Page

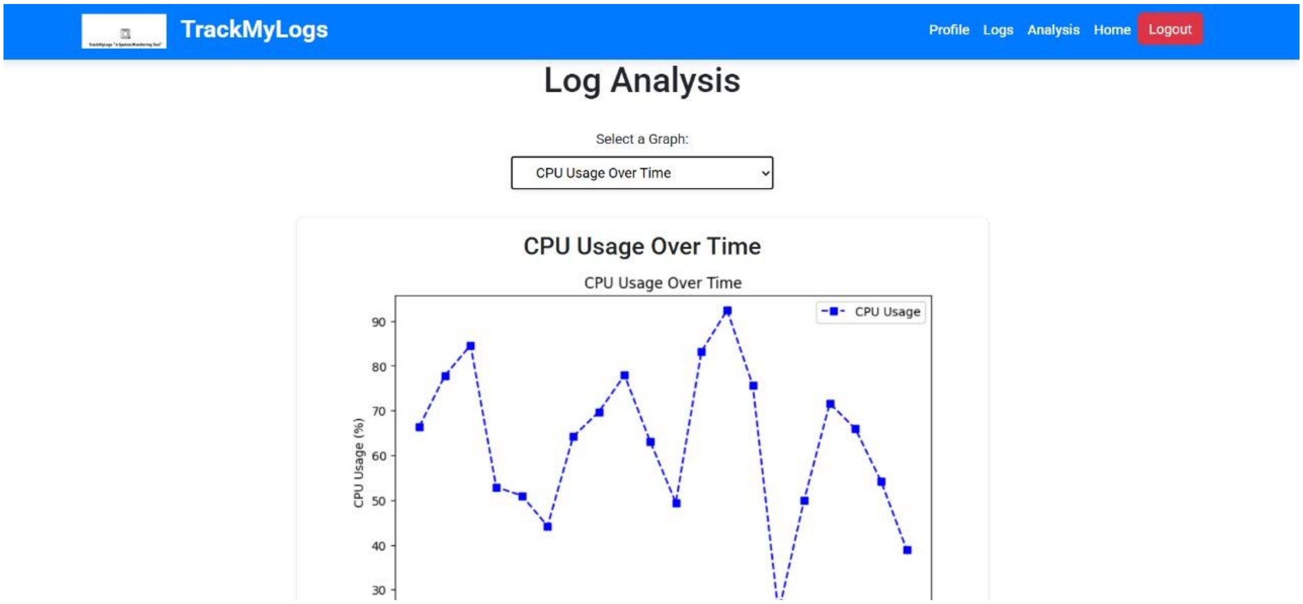

Figure 3.5 Analysis Page

- The **Analysis Page** transforms raw log data into insightful visualizations. Users are presented with a set of **interactive charts and graphs**. These visualizations include:

➢ Frequency of system/application events
➢ Error occurrence trends
➢ User activity graphs
➢ Anomalies or spikes in usage

- From this page, users can also **export the analyzed logs into a PDF report**, which is automatically emailed to their registered account. This report serves as a portable, shareable version of their log analysis.

3.3 Testing using Use Cases

3.3.1 Registration (Sign Up):

Test Case	Test Data	Test Result	Test Report
Blank email, password, confirm password	Email: - Password:- Confirm Password: -	Invalid	Please enter the details
Existing email, password, confirm password	Email: admin@gmail.com Password: 98766 Confirm Password: 98766	Invalid	Email id already registered
Valid email, password and confirm password	Email: abc@gmail.com Password: user123 Confirm Password: user123	Valid	Welcome message

Table 3.1 Testing Report Registration (Sign Up)

3.3.2 Login

Test Case	**Test Data**	**Test Result**	**Test Report**
Blank email,password	Email: Password :	Invalid	Please enter the details
Invalid email or password	Email: xyz@gmail.com Password: 676767	Invalid	Email id or Password is incorrect
Valid email,password	Email:abc@gmail.com Password: user123	Valid	Welcome Page

Table 3.2 Testing Report Login

3.3.3 Forgot Password

Test Case	Test Data	Test Result	Test Report
Blank or invalid email	Email:xyz@gmail.com	Invalid	Given email is not registered
Valid Email	Email:abc@gmail.com	Valid	The otp will be sent to the given email
Valid otp	Otp:123456	Valid	Redirect to forgot password page
Invalid otp	Otp:801827	Invalid	Incorrect otp,enter otp again
Blank password and conform password	Password: Conform password:	Invalid	Enter proper password
Password mismatch	Password:123 Conform password:456	Invalid	Password did not match
Password matched	Password:123 Conform password:123	Valid	Password Changed and redirected to login page

Table 3.3 Testing Report Forgot Password

3.3.4 Download Desktop App

Test Case	Test Data	Test Result	Test Report
Terms And Condition to Download App	Uncheck	Invalid	Download Button Disable
Terms And Condition to Download App	Check	Valid	Download button Enable

Table 3.4 Download Desktop App

3.3.5 Authorization Token

Test Case	Test Data	Test Result	Test Report
Token Generation	Token Previously Generated	Invalid	Token Will not Generate
Token Generation	Token Previously Not Generated	Valid	Token Will Generate

Table 3.5 Testing Report Authorization Token

3.3.6 Desktop App Authentication

Test Case	Test Data	Test Result	Test Report
Authorized email, Token	Email: - Token: -	Invalid	Please enter the details
Invalid email or Token	Email:user2@mail.com Token: 394e	Invalid	Email id or Token is incorrect
Authorized Email and Token	Email:user2@email.com Token : 394e	Valid	Log Generation will be started

Table 3.6 Testing Report Desktop App Authentication

3.3.7 Types of Log generation

Test Case	Test Data	Test Result	Test Report
System Information Logs	Desktop App Fetches System Information	Invalid	Failed to get System Information
System Information Logs	Desktop App Fetches System Information	Valid	Get The System Information
Directory Monitoring Information	User Modify or Create any File or Folder within same Directory	Invalid	Failed to get Directory Monitoring Information
Directory Monitoring Information	User Modify or Create any File or Folder within same Directory	Valid	Get the Directory Monitoring Information
OS Information	Desktop App Fetches OS Information	Invalid	Failed to get OS Information
OS Information	Desktop App Fetches OS Information	Valid	Get The OS Information
Opened App	User opens Any Application	Invalid	Failed to get Opened App Information
Opened App	Users opens Any Application	Valid	Get The Opened App Information
Network Activity Information	Desktop App Fetches Network Activity Information	Invalid	Failed to get Network Activity Information

Table 3.7 Testing Report Types of Logs Generation

Chapter 4: Conclusion and Future work

4.1 Conclusion

The TrackMyLogs project successfully implements a web-based log monitoring and visualization system that collects system logs from client machines, temporarily stores them using Redis, and then processes them using a Django-based backend. The system performs automated log analysis to detect meaningful patterns, generate summarized reports, and visualize data in a user-friendly format. Through this platform, users can efficiently track and diagnose system behavior, enhancing both system performance and security monitoring. The project's modular architecture and use of open-source technologies ensured ease of development, maintainability, and scalability.

4.2 Future work

To further enhance the TrackMyLogs system, the following improvements and features can be considered in the future:

- **Alert System**: Introduce a threshold-based or anomaly detection alert mechanism to notify users of unusual activity via email or dashboard notifications.
- **Cloud Storage Integration**: Enable storing logs in cloud platforms like AWS S3 or Azure Blob for better scalability and reliability.
- **Advanced Analytics with ML**: Integrate machine learning algorithms to classify logs, detect anomalies, or predict failures.
- **Dockerization and Deployment**: Package the application using Docker and deploy it on cloud infrastructure for broader availability and testing.
- **Dark Mode & Theme Customization**: Enhance user experience by allowing UI customization.

Web References

Websites Used as References

- https://www.djangoproject.com/ – Official site for Django framework
- https://www.sqlite.org/ – SQLite documentation and features
- https://redis.io/docs/ – Official Redis documentation
- https://matplotlib.org/ – Data visualization in Python
- https://docs.python.org/3/ – Python official documentation
- https://getbootstrap.com/ – UI framework documentation

Literature References

Research Papers and Other Materials

[1] A. Sharma and N. Gupta, *"Log Management: Challenges and Solutions,"* International Journal of Information Technology and Computer Science, vol. 12, no. 4, pp. 45–52, 2022.
[2] R. Verma and S. Thakur, *"Secure Log Transfer Protocols: A Comparative Study,"* Journal of Cybersecurity and Data Protection, vol. 9, no. 1, pp. 12–19, 2021.
[3] M. Roy and P. Nair, *"User-Centric Design in Log Management Systems,"* Human-Centric Computing and Information Sciences, vol. 7, no. 3, pp. 55–68, 2023.
[4] T. K. Sen and D. K. Patel, *"Analyzing Log Data: Techniques and Tools,"* IEEE Access, vol. 11, pp. 7894–7907, 2023.
[5] J. Thomas and R. Iyer, *"Role-Based Access Control in Log Management,"* ACM Transactions on Information and System Security (TISSEC), vol. 19, no. 2, pp. 1–18, 2020.